

Week 2: Embeddings, Vector Stores, and RAG

AI Engineering Internship · build first, theory only as needed · free-tier stack

This week you're building the thing everyone throws around as "RAG" without really knowing what's inside it. By the end you'll have a CLI tool that reads a real document and answers questions grounded in it, not just whatever the model remembers from training. Same rule as last week, read the "what it means" part, google around it, then build.

WEEK PLAN

Day by day this week

- **Day 6:** embeddings, load a model, compute similarity scores, build `most_similar()`.
- **Day 7:** chunking, split a real document, manually check the chunks make sense.
- **Day 8:** vector store, set up Chroma or FAISS, store chunks, build top-k retrieval.
- **Day 9:** RAG v1, wire retrieval into your chatbot, test grounded answers and the "I don't know" case.
- **Day 10:** polish, add the citation feature, then the Friday sync and quiz.

The topics below go into more depth on each of these, use this list just to know where you should roughly be on a given day.

One setup note before you start: OpenRouter and Groq only do chat, they don't have an embeddings endpoint. So for embeddings this week you'll run a small model locally on your own machine using the `sentence-transformers` library. It's free, it's fast enough for what we're doing, and it means you're not blocked by any API limits for this part.

TOPIC 1

Topic 1: Embeddings

What it covers: turning text into numbers in a way that captures meaning, not just characters.

What it means: an embedding is a list of numbers (a vector) that represents a piece of text. The trick is that texts with similar meaning end up with similar vectors, even if the words used are completely different. "The cat sat on the mat" and "a feline rested on the rug" would land close together in this vector space, while "the cat sat on the mat" and "quarterly revenue projections" would land far apart. Once you have vectors, you can measure how close two pieces of text are using cosine similarity, which is just a number between -1 and 1 telling you how aligned two vectors are. Don't overthink the math behind cosine similarity, just know it gives you a score where higher means more similar.

What to google:

- "what is a text embedding"
- "sentence-transformers python quickstart"
- "cosine similarity explained"

What to build:

- Load a sentence-transformers model (all-MiniLM-L6-v2 is small, fast, and good enough to start with) and embed 5-6 sentences you write yourself, mixing some similar ones and some unrelated ones.
- Compute cosine similarity between every pair and print it as a little table. Check that similar sentences actually score higher than unrelated ones, if they don't, something's off in your setup.
- Write a tiny function `most_similar(query, sentences)` that takes a new sentence and tells you which of your original 5-6 it's closest to.

Something extra: look up "embedding dimensions", notice the model you used outputs vectors of a fixed size (384 for MiniLM). Bigger embedding models generally capture more nuance but cost more to run and store. You don't need a bigger one for this internship, just know that tradeoff exists.

TOPIC 2

Topic 2: Chunking

What it covers: breaking a long document into pieces small enough to embed and search properly, instead of trying to embed the whole thing at once.

What it means: you can't just take a 50-page PDF, turn it into one embedding, and expect to answer specific questions about page 30. Embeddings work best on chunks of a reasonable size, maybe a few hundred words each. The simplest approach is fixed-size chunking, just cut the text every N characters or words, sometimes with a bit of overlap between chunks so you don't cut a sentence in half and lose context. A better approach is semantic chunking, splitting on natural boundaries like paragraphs or sections instead of a raw character count. Start with fixed-size, it's good enough to learn the concept, semantic chunking is the "something extra" here.

What to google:

- "text chunking for rag"
- "chunk overlap explained"
- "recursive character text splitter"

What to build:

- Take any PDF or long text file (a manual, an article, whatever you have lying around) and write a function that splits it into chunks of roughly 300-500 words, with maybe 50 words of overlap between consecutive chunks.
- Print out the first few chunks and manually check they make sense, that you're not cutting off mid-sentence in a way that loses the point.
- Try changing your chunk size smaller and bigger, and just notice how the chunks read differently. You don't need to write this up anywhere, just build a feel for it.

Something extra: look up "semantic chunking" or "recursive character text splitter" if you want to try splitting on paragraph breaks or headings instead of a fixed character count. Not required this week.

TOPIC 3

Topic 3: Vector stores and retrieval

What it covers: storing your chunk embeddings somewhere searchable, and pulling back the most relevant ones for a given question.

What it means: a vector store is basically a database built for one thing, given a query vector, find the stored vectors closest to it, fast. You'll use Chroma or FAISS, both are free, both run locally, no server setup needed for what we're doing. The workflow is: embed all your chunks, store them with their embeddings, then when a question comes in, embed the question the same way, and ask the store for the top-k closest chunks (k is just how many you want back, usually 3-5). This is where things quietly break in real projects, if your chunking was bad, or k is too small or too big, your retrieval brings back the wrong stuff and the model answers from garbage. Worth testing on purpose.

What to google:

- "chromadb python quickstart"
- "faiss python quickstart"
- "top k similarity search"

What to build:

- Pick either Chroma or FAISS (Chroma is a bit easier to start with, don't stress about picking the "better" one). Store the chunks from Topic 2 along with their embeddings.
- Write a function that takes a question, embeds it, and returns the top 3-5 closest chunks with their similarity scores.
- Test it with a question you know the document answers well, and a question you know the document doesn't cover at all. Look at what comes back in both cases.

Something extra: try changing k from 3 to 10 on the same question and see how the retrieved chunks change. Also worth trying, deliberately pick a badly worded or vague question and see how much retrieval quality drops.

TOPIC 4

Topic 4: Putting it together, RAG v1

What it covers: connecting last week's chatbot script to this week's retrieval, so the model answers using the document instead of guessing.

What it means: RAG (retrieval augmented generation) is just this pattern, take the user's question, retrieve the most relevant chunks, stuff those chunks into the prompt along with the question, then let the model answer using only what's in front of it. The important part is the prompt has to actually tell the model to stick to the provided context and say it doesn't know if the answer isn't there, otherwise it'll happily make something up that sounds right.

What to google:

- "retrieval augmented generation explained"
- "rag prompt template grounding"

What to build:

- A CLI tool: ask a question, it retrieves the top chunks from Topic 3, builds a prompt with those chunks plus the question, sends it to the model through OpenRouter or Groq, prints the answer.
- Test it with a question the document answers clearly, check the answer is actually grounded in the text and not just something the model already knew.
- Test it with a question the document doesn't cover. If the model confidently makes something up instead of saying it doesn't know, that's a prompt problem, go back and add an explicit instruction like "only answer using the context below, if the answer isn't there say you don't know."

Something extra: print out which chunks got used for a given answer alongside the answer itself. This is basically a citation feature, and it's a small thing that makes a RAG tool feel a lot more trustworthy and easier to debug.

FRIDAY CHECKPOINT**By Friday**

You should have a CLI tool that takes a real document (something with actual content, not a paragraph you wrote for testing) and answers questions grounded in it, including saying "I don't know" when it should. Come to the sync ready to demo it live, including at least one question that trips it up, and be ready to explain what embeddings and chunking actually are without reading off a definition.